

Learning Association-Line Transformations over Autosegmental Graphs

1. Introduction

This paper presents a method for learning transformations over multi-linear structures, with a particular focus on sequences which are synchronized in time. Multi-linear structures are widely used in linguistics in phonological theory in particular, and are known as *Autosegmental Representations* (ARs; Clements and Goldsmith, 1984; Goldsmith, 1976; Hayes, 1980; Sagey, 1986; Jardine, 2017). ARs are graph structures which consist of one or more sequences called ‘tiers’ and cross-tier association relations. This work focuses on a specific, but important, type of transformation where the association lines are modified. We formalize these transformations as graph transductions (Courcelle, 1994; Engelfriet and Hoogeboom, 2001; Courcelle and Engelfriet, 2012) over ARs (Jardine and Heinz, 2015; Jardine, 2017; Strother-Garcia, 2019) and use a bottom-up factor inference method (Chandlee et al., 2019; Rawski, 2021; Yolyan, 2025a) to infer which association lines are preserved, deleted, or newly introduced from observed input-output pairs.

Multi-linear structures have been widely used in linguistic theory, most notably in autosegmental phonology, to represent different levels of representations that are coordinated in time, including tone ¹(Goldsmith, 1976), syllable structure (Clements and Keyser, 1983), prosodic structure (Hayes, 1980), and feature geometry (Sagey, 1986). The core idea is that elements on different tiers are temporally synchronized via an association relation. Many input-output mappings are naturally characterized by rearranging those association lines across tiers (Goldsmith, 1976; Clements and Goldsmith, 1984; Hyman, 2009).

For example, one instance of reduplication in Estako (Elimelech, 1976), a tonal Niger-Congo language, is shown in Figure 1. Segmental (whether consonant or vowels) and tonal information are represented on separate tiers, but are coordinated in time by the association relation (shown in figures in this paper as lines). The word for ‘house’ in Estako is pronounced [ówà], with a high (H) tone on the first vowel, marked by an acute accent (´), and a low (L) tone on the second vowel, marked by a grave accent (`). In Estako, reduplication conveys a repetitive meaning of ‘every X’. Interestingly, the reduplicated form of [ówà] ‘house’ is not [ówàówà] as may be expected, but in fact [ówǒwà], with high, rising, and low tones associated with the three vowels, respectively. In many languages including Estako, when reduplication creates a sequence of adjacent vowels V_1V_2 , the first vowel V_1 deletes but its associated tone does not. Instead, it reassociates to the following vowel V_2 , and produces a rising tone on V_2 in the output, as shown in Figure 1.

A large body of theoretical work on African tonal processes has demonstrated the advantages of autosegmental graphs (Clements and Goldsmith, 1984; Odden, 2013; Hyman,

1. Tone refers to the pitch level of a vowel or syllable. Languages which use pitch to mark lexical contrasts are called tonal languages. English is not a tonal language, whereas Mandarin contrasts words such as *mā* ‘mother’ and *mǎ* ‘horse’ solely by tone.

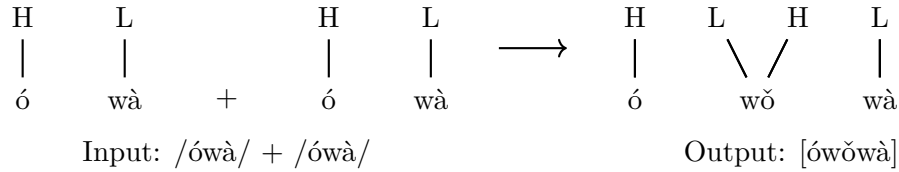


Figure 1: Association transformation in /ówà/ ‘house’ + /ówà/ ‘house’ $\rightarrow$ [ówǒwà] ‘every house’. Deletion of the stem-final vowel triggers association change.

2009). Computationally, using multi-tiered structures has been shown to reduce the complexity of tonal patterns (Jardine, 2016; Chandlee and Jardine, 2021a). Previous work has typically reduced these structures to string representations, either through synchronized tapes or by hard-coding association types (Wiebe, 1992; Bird and Ellison, 1994; Yli-Jyrä, 2013, 2015; Rawski and Dolatian, 2020).

In contrast, we propose an approach that learns the association transformations directly over graphs. As a preliminary study, we restrict our attention to tonal processes involving a single deassociation and/or reassociation operation.

We adopt the model-theoretic logical transduction framework of Yolyan (2025a). Yolyan synthesizes insights from graph transductions (Courcelle, 1994; Engelfriet and Hoogboom, 2001; Courcelle and Engelfriet, 2012) with learning of classes of formal languages using model-theoretic representations (Strother-Garcia et al., 2016; Vu et al., 2018; Chandlee et al., 2019) to learn a class of string-to-string functions. The transductions specify, via a collection of logical formulas, how to construct an output structure from a given input structure. Yolyan searches for a minimal distinguishing structure based on the Bottom-Up Factor Inference Algorithm (BUFIA; Chandlee et al., 2019; Rawski, 2021; Li, 2025; Swanson et al., 2026; Li and Heinz, to appear). BUFIA searches a partially ordered space of model-theoretic structures to identify a minimal hypothesis consistent with the data. For transformation learning, Yolyan (2025a,b) uses this idea to find a minimal structure which distinguishes positive examples (those that undergo the change) from negative examples (those that do not).

This work brings together these representational and learning perspectives. Using a case study of tonal contour formation (as shown in Figure 1), we show how logical transductions over autosegmental graphs can be learned using a BUFIA-style learner. In contrast to Yolyan’s previous work on string functions, the target of learning here is how association relations change. While the case study is drawn from tonal phonology, the same approach can in principle be extended to other multi-linear structures. To our knowledge, the learnability of such transformations has not previously been investigated.

2. Preliminaries

This section provides background on autosegmental graphs, matrix and logical representations thereof. We assume familiarity with first order logic.

ARs represent the idea that different aspects of linguistic structure are organized on distinct levels while progressing simultaneously over time. Each tier encodes a particular type of information. In this work, we assume a two-tiered structure: a tonal tier with two

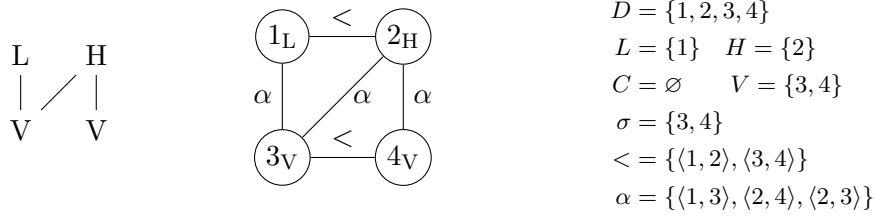


Figure 2: Three representations of a two-tier autosegmental graph.

tones, H and L; and a segmental tier with two segment types, consonants (C) and vowels (V), though the definition remains the same in the general case.

We define ARs using model-theoretic relational structures (Jardine, 2017). A relational structure contains a domain D and instantiates a set of relations over that domain. The set of relations is called the *signature* of the model. For concreteness, in this work, the signature of the model contains the unary relations $\{L, H, C, V\}$ indicating which domain elements are low tones, high tones, consonant and vowels. The L and H make up the tonal tier and C and V make up the segmental tier. The signature also contains two binary relations: $<$ is the precedence relation and α is the association relation. Well-formed ARs obey the following constraints:

- The domain is partitioned into two sets: T (tonal tier) and S (segmental tier).
- Every element in T satisfies exactly one of L or H and nothing else.
- Every element in S satisfies exactly one of C or V and nothing else.
- If $<(x, y)$ (equivalently $x < y$) is satisfied then either $x \in T \wedge y \in T$ or $x \in S \wedge y \in S$. Also, for each tier, the precedence relation produces a total ordering over it.
- If $\alpha(x, y)$ is satisfied then either $x \in T \wedge y \in S$ or $y \in T \wedge x \in S^2$.
- For all $(x, y), (x', y') \in \alpha, x < x' \Rightarrow (y < y')(y = y')$. Informally, two association lines do not cross, a condition known as No-Crossing Constraint (NCC, Goldsmith, 1976; Coleman and Local, 1991)

In short, an AR is a graph, whose vertices are partitioned into two disjoint sets, with each tier linearly-ordered, whose elements are coordinated via a temporally well-behaved association relation³. Figure 2 shows an AR on the left, its corresponding model-theoretic structure on the right, and the graph of the model in the center. Henceforth, we use the words ‘node’ and ‘domain element’ interchangeably.

The *size* of an AR is defined as the size of the domain plus the size of the unary relations and half the size of the association relation. We leave out the precedence relation because its size is fixed given the lengths of each tier. We divide by two because α is symmetric. Formally, the size of an AR is $|D| + |H| + |L| + |C| + |V| + |\alpha|/2$. Thus, the size of the AR in Figure 2 is 11.

2. The association relation is symmetric, although symmetry is omitted from figures and logical formulas for readability. Hereafter, we use x to denote tonal elements and y to denote over segmental elements.

3. It is straightforward to generalize ARs to include n -partite graphs with n tiers.

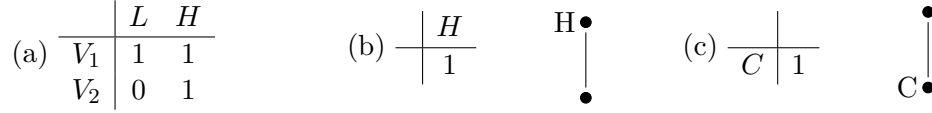


Figure 3: Three matrix representations. The one in (a) is equivalent to the AR in Figure 2. The (partial) ARs in (b) and (c) are equivalent to the graphs to their right.

Just as it is convenient to view ARs as graphs, it is also useful to represent them with *adjacency matrices*, which use binary values to denote whether there is an edge between two nodes. The indices along the rows and columns indicate the linear order of elements on the tonal and segmental tiers. For example, Figure 2a. Formally, the associations of AR $\mathcal{A} = \langle D, L, H, C, V, \alpha, < \rangle$ can be represented by

$$M_{[x,y]} = \begin{cases} 1 & \text{if } (x, y) \in \alpha, \\ 0 & \text{otherwise,} \end{cases}$$

The motivation for adopting matrices is convenience. It will be important to identify structures contained within other structures. When working with strings, notions of subsequence and substring are well-defined. While [Chandlee et al. \(2019\)](#) and [Rogers and Lambert \(2019\)](#) define parallel notions for arbitrary relational structures, which can also be applied to ARs, we find it simpler to develop these notions for ARs in the context of matrices. Let B be a matrix and A be a $m \times n$ matrix. Informally, A is a *submatrix*⁴ of B if it can be formed by taking $(1 \leq k \leq m)$ contiguous rows and ℓ $(1 \leq \ell \leq n)$ contiguous columns from B , and the corresponding row and column labels are consistent.

Formally, an $m \times n$ matrix A is contained in a matrix B , denoted $A \sqsubseteq B$, if there exists an $m \times n$ submatrix of B , and bijections r, c , denoting the row and column bijections, which map indices in A to indices in B such that for all $i \in [1, m]$ and $j \in [1, n]$ the following hold:

1. $r(i) = x$ and $c(j) = y$, and
2. $B_{xy} \geq A_{ij}$, and
3. $V(i) \Rightarrow \neg C(r(i))$, $C(i) \Rightarrow \neg V(r(i))$, and
4. $H(j) \Rightarrow \neg L(r(j))$, $L(j) \Rightarrow \neg H(r(j))$.

For example, the matrix in Figure 3b is contained within the matrix in Figure 3 as witnessed by the bijections $r(1) = 1$, $c(1) = 2$. On the other hand, the matrix in Figure 3c is not contained within the matrix in Figure 3 because $C(1) \Rightarrow \neg V(r(i))$ and for any bijection r , $V(r(1))$ is true.

A matrix A is a *subfactor* of B if and only if A is contained in B . Conversely, B is a *superfactor* of A . Furthermore, if the size difference between a superfactor and its subfactor is exactly one, the superfactor is called the *next superfactor* of that subfactor.

There is a general connection between connected structures, logical formulas, and containment. For example, the structure in Figure 2 corresponds to the logical formula:

$$\exists w, x, y, z (L(w) \wedge H(x) \wedge V(y) \wedge V(z) \wedge (w < x \wedge y < z) \wedge \alpha(w, y) \wedge \alpha(x, z) \wedge \alpha(x, y)).$$

4. The term “submatrix” is generally defined by deleting arbitrary rows and/or columns from the original matrix. However, in our case, we require the remaining rows and columns must be *contiguous* in the original matrix, which is related to the notion of *connectedness* in [Chandlee et al. \(2019\)](#).

Operation	Matrix	Graph	Logic structure(x, y)	Size
Initial	$\begin{array}{c c} & x \\ \hline y & 1 \end{array}$	$\begin{array}{c} y \bullet \\ \\ x \bullet \end{array}$	$\alpha(x, y)$	3
Add label	$\begin{array}{c c} & H \\ \hline y & 1 \end{array}$	$\begin{array}{c} H(y) \bullet \\ \\ x \bullet \end{array}$	$\alpha(x, y) \wedge H(y)$	4
Add node	$\begin{array}{c cc} & H & z \\ \hline y & 1 & \end{array}$	$\begin{array}{c} H(y) \bullet \\ \\ x \bullet \end{array} \quad \bullet z$	$\alpha(x, y) \wedge H(x) \wedge \exists z(x \triangleleft z)$	5
Add association	$\begin{array}{c cc} & H & z \\ \hline y & 1 & 1 \end{array}$	$\begin{array}{c} H(y) \bullet \\ \quad \diagdown \\ x \bullet \quad \bullet z \end{array}$	$\alpha(x, y) \wedge H(x) \wedge \exists z((x \triangleleft z) \wedge \alpha(y, z))$	6

Table 1: Examples of operations used in generating NEXTSUPFACT hypotheses across matrix, graph, and logical representations.

Any structures that satisfy this formula contains a structure isomorphic to the one in in Figure 2.

Table 1 shows matrix, graphical, and logical representations of different structures where the the variables x, y (which are in scope by the predicate $\text{structure}(x, y)$). Furthermore, pairs of adjacent rows in Table 1 stand in the next superfactor relationship. The structure denoted by $\alpha(x, y) \wedge H(y)$ is a next superfactor of $\alpha(x, y)$ and so on. The ‘Operation’ column indicates the change that was made to obtain the next superfactor from the structure in the previous row. Note we also use the successor relation $\triangleleft$, which is the transitive reduction of $<$: $x \triangleleft y$ if and only if $x < y \wedge \neg(\exists z)[x < z < y]$. We make use of both precedence and successor as needed.

This containment relation provides a *partial ordering* over the space of ARs. We illustrate this partial order with strings in Figure 4. A superfactor is connected to its subfactors, which are positioned below it in the diagram. The partial ordering in turn leads to entailment relations. If a string contains some factor s , then it contains all subfactors of s . Similarly, if it does not contain s , then it does not contain any superfactor of s . Readers are referred to related work (Chandlee et al., 2019; Rawski, 2021; Swanson, 2025; Swanson et al., 2026; Payne, 2024; Li, 2025) for a more detailed discussion of the partial ordering itself. Here, we emphasize that this partial order forms the foundation of bottom-up learning algorithms, since it provides a principled way to navigate the hypothesis space through entailment relations between factors.

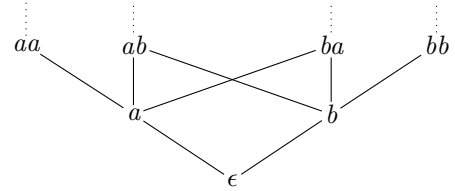


Figure 4: Strings over $\Sigma = \{a, b\}$ ordered by the containment relation (non-exhaustive)

[I think we should add a paragraph here about BUFIA, it's learning guarantees, and its time complexity.]

x3

3. Graph Transductions

We model a phonological process with graph transductions from input structures to output structures (Courcelle, 1994; Engelfriet and Hoogetboom, 2001; Courcelle and Engelfriet, 2012). Such transductions have been used to model phonological processes (Strother-Garcia, 2018; Dolatian, 2020; Chandlee and Jardine, 2021b), as well as to study different representations (Strother-Garcia, 2019; Oakden, 2020; Jardine et al., 2021; Danis, 2025).

In this framework, a set of logic formulas define a function from ARs to ARs. The formulas are always evaluated with respect to the input structure. Whether they evaluate to true or false has consequences for the structure of the output. Table 2 provides the formulas described below.

- The domain formula ϕ_{Domain} determines which ARs are in the domain of the function. Here, we set this value to True, indicating this function is total.
- The copy set helps determine the size of the output structures. Here, we set it to 1, which means every node in the output corresponds to one element in the input.
- For each unary relation $P \in \{L, H, C, V\}$, the formula $\phi_P(x)$ determines whether node x in the output has property P . Here, nodes in the output have the same properties as their corresponding nodes in the input.
- The formula $\phi_{<}(x, y)$ determines which pairs of nodes in the output structure stand in the precedence relation. Here, pairs of nodes in the output stand in the precedence relation only if their corresponding nodes in the input are in the precedence relation.
- The formula $\phi_{\alpha}(x, y)$ determines which pairs of nodes in the output structure stand in the association relation. Here pairs of nodes stand in the association relation provided (i) they were associated in the input and they do not satisfy the deassociation condition **deassoc** or (ii) they were not associated in the input and they do satisfy the reassociation condition **reassoc**. We refer to these two components as **Retain** and **Add**, respectively. The predicates **deassoc** and **reassoc** are language-specific logical formulas, which correspond to structures, and which constitute the parameters to be learned.

$$\phi_{\alpha}(x, y) := \underbrace{(\alpha(x, y) \wedge \neg \text{deassoc}(x, y))}_{\text{Retain}} \vee \underbrace{(\neg \alpha(x, y) \wedge \text{reassoc}(x, y))}_{\text{Add}} \quad (1)$$

- The licensing formula $\phi_{\text{license}}(x)$ determines which nodes in the output are licensed. According to the semantics of graph transductions, unlicensed nodes disappear, along with any relations associated with them. Here, only output nodes that stand in an association relation in the output are licensed.

It follows that the transduction defines an output structure whose domain is the same as the input structure, except that unassociated elements are filtered out by $\phi_{\text{license}}(x)$. All unary relations $P(x)$ in the input signature are preserved in the corresponding output.

Output	Input	Interpretation
ϕ_{Domain}	$:= \text{TRUE}$	Preserve all input elements
COPY	$:= 1$	Define output domain same size as input
$\phi_P(x)$	$:= P(x)$	Preserve elements and their labels
$\phi_{<}(x, y)$	$:= <(x, y)$	Preserve linear order within tiers
$\phi_{\alpha}(x, y)$	$:= \begin{aligned} &(\alpha(x, y) \wedge \neg \text{deassoc}(x, y)) \\ &\vee (\neg \alpha(x, y) \wedge \text{reassoc}(x, y)) \end{aligned}$	Update association relations
$\phi_{\text{license}}(x)$	$:= \exists y \phi_{\alpha}(x, y)$	Preserve only associated elements

Table 2: Logical formulas defining an input-output AR transduction.



Figure 5: Contour formation in both graph-theoretic and matrix representations.

Similarly, precedence relations are preserved. The only change is the association relation α . In this way, the problem of learning association-line transformations can be reduced to learning the logical formulas for **deassoc** and/or **reassoc**, or their equivalent structural representations.

When no transformation occurs between the input and output representations, neither **deassoc** nor **reassoc** takes place (i.e., both have the value of False). The association relation therefore remains unchanged: all input associations are preserved in the output (**Retain** evaluates to True iff $\alpha(x, y)$ holds in the input structure), and no new associations are added (**Add** applies to none because it always evaluates False). Since these two predicates are combined through disjunction, two parallel learning processes apply simultaneously, one **deassoc** and one for **reassoc**, and their outputs are combined to define the final association relation ϕ_{α} .

To illustrate, Figure 5 visualizes this contour formation from Figure 1 in both graph-theoretic and matrix representations. The **deassoc** function identifies the association to be broken between T_1 and the deleted vowel V_1 (the crossed-out line). The **Retain** conjunction $\alpha(x, y) \wedge \neg \text{deassoc}(x, y)$ removes this association (T_1, V_1) while preserving all unaffected associations, that is, (T_2, V_2) . The **reassoc** function then links T_1 to the surviving vowel V_2 (the dotted line). As a result, V_1 is no longer associated with any element on the opposite tier and therefore fails to satisfy the licensing condition ϕ_{license} in the output. Consequently, V_1 is not licensed and is omitted from the surface form (shown in gray).

The **deassoc** and **reassoc** operations are equivalently reflected in the adjacency matrices in Figure 5. The value for (T_1, V_1) changes from 1 (in the input) to 0 (in the output), which indicates delinking; the value corresponding to (T_1, V_2) changes from 0 to 1, which indicates reassociation. Formally, the set of deassociated pairs and the set of reassociated pairs can

be identified by Equations 2 and 3, respectively.

$$D(UR, SR) = \{(i, j) \mid SR_{ij} < UR_{ij}\} \quad (2)$$

$$R(UR, SR) = \{(i, j) \mid UR_{ij} < SR_{ij}\} \quad (3)$$

4. Association Learning

This section follows the ideas of using logical transductions to learn phonological mappings in Yolyan (2025a), who in turn leveraged the entailment relation in the partially ordered space crucial to Chandlee et al. (2019) to find a smallest environment that triggers a change.

Importantly, if an environment (which itself is a structure defined model-theoretically, henceforth we call it ‘environment’) is sufficient to trigger a transformation, then any superfactor containing that environment will likewise undergo the same transformation. Conversely, if an environment fails to trigger the change, then none of its subfactors can do so. Given a hypothesized environment, all data satisfying this condition are expected to undergo the transformation (positive evidence), while all data not satisfying this condition are expected not to undergo the transformation (negative evidence). If some positive data fails to undergo the transformation, then this hypothesis is too restrictive; if some negative data nevertheless undergo the transformation, then the hypothesis is too general. The goal of learning is therefore to identify the smallest common environment that is contained in all positive evidence, but absent from all negative evidence.

Yolyan (2025a) worked with string-to-string functions. However, for ARs, environments are expressed as structural relations over graphs. Because model-theoretic representations generally form a partial order with the containment relation (Chandlee et al., 2019), the same ideas apply to ARs.

The core learning algorithm is shown in Algorithm ?? (BUFIA-AC). It takes as input a set of input-output pairs, an initial hypothesized environment H_0 , and a bound $k \in \mathbb{N}$ on the size of environments to consider. To find the environment for **Retain**, BUFIA-AC is called with the most general hypothesis $h_0 = \alpha(x, y)$. To find the environment for **Add**, BUFIA-AC is called with $h_0 = \neg\alpha(x, y)$. Thus BUFIA-AC is called twice to learn the two environments independently. Those results are combined through disjunction to obtain the formula in Equation 1 as shown in Algorithm ??.

The input data consists of a finite set of input-output AR pairs, which are partitioned into positive and negative evidence, denoted D^+ and D^- respectively. It also maintains a pruning set P , and a queue, Q of hypotheses to be checked. The queue is initialized with the structure c_0 as the initial hypothesis.

The algorithm conducts a breadth-first search over a partially ordered hypothesis space. At each step, a candidate hypothesis h is first tested against the positive evidence. For each $(I, O) \in D^+$ $\text{MATCH}(h, I, O, h_0)$ returns true iff the hypothesized environment h matches the relevant positions in the structure that undergo the transformation.

Formally, this is carried about by Algorithm ?? (Match). It uses h_0 to determine whether it is looking for deassociations or reassociations and behaves accordingly. In the former case, it checks whether there are bijections r, c such that $(r(x), c(y)) = D(I, O)$ for each $(I, O) \in D^+$, and that these bijections witness h being contained in I . These conditions ensure that the x, y coordinates are anchored to the positions in the structure that undergo

Algorithm 1: BUFIA-AC

Input: $D = \{(I_1, O_1), \dots, (I_n, O_n)\}, h_0, k$
Output: An environment c or $\perp$
 $D^+ \leftarrow \{I_i \mid I_i \neq O_i\}$
 $D^- \leftarrow \{I_i \mid I_i = O_i\}$
 $P \leftarrow \emptyset$
 $Q \leftarrow h_0$
while $Q \neq \emptyset$ **do**
 $h \leftarrow Q.\text{Dequeue}()$
 if $\forall I_i \in D^+ [\text{MATCH}(h, I_i, O_i, h_0)]$ **then**
 if $\forall I_i \in D^- [\neg \text{MATCH}(h, I_i, O_i, h_0)]$ **then**
 return h
 else
 foreach $s \in \text{NEXTSUPFACT}(h)$ **do**
 if $|s| < k \wedge (\forall p \in P)[s \not\sqsubseteq p]$ **then**
 $Q.\text{Enqueue}(s)$
 else
 $P \leftarrow P \cup \{h\}$
return $\perp$

Algorithm 2: Learning ϕ_α (LPA)

Input: $D = \{(I_1, O_1), \dots, (I_n, O_n)\}, k$
Output: The formula $\phi_\alpha(x, y)$ or $\perp$
 $\text{Retain} \leftarrow \text{BUFIA-AC}(D, \alpha(x, y), k)$
 $\text{Add} \leftarrow \text{BUFIA-AC}(D, \neg \alpha(x, y), k)$
if $\text{Retain} = \perp \vee \text{Add} = \perp$ **then**
 return $\perp$
else
 return $\text{Retain} \vee \text{Add}$

Algorithm 3: Match

Input: h, I, O, h_0
Output: True or False
if $h_0 = \alpha(x, y)$ **then**
 if $\forall (i, j) \in D(I, O)$
 $[\exists r, c : r(x) = i, c(y) = j, h \sqsubseteq I]$ **then**
 return *True*
 else
 if $\forall (i, j) \in R(I, O)$
 $[\exists r, c : r(x) = i, c(y) = j, h \not\sqsubseteq I]$ **then**
 return *True*
 return *False*

the transformation. The reassociation case proceeds similarly. If h fails to match all positive examples D^+ , then the current hypothesis is incompatible with the observed transformation and thus is added to the pruning set P . Because any superfactor of h will also fail to be contained in the same positive example, such hypotheses need not be refined further.

If h passes the positive test, it is then tested against all negative examples in D^- . Then h is checked to see whether it matches any datum in D^- . If not, it is consistent with both the positive and negative evidence. Therefore h is returned as a valid environment, and BUFIA-AC terminates. Otherwise, h is too general. In this case, the learner generates the next superfactors of h using NEXTSUPFACT and adds them to the queue, provided that they do not contain a pruned hypothesis and do not exceed the complexity bound k .

Since hypotheses are explored breadth-first, the first valid hypothesis returned by the learner is guaranteed to be a minimal structural environment consistent with all positive and negative evidence.

In terms of NEXTSUPFACT, there are many possible ways to generate superfactors while still covering the same hypothesis space. Importantly, each operation increases the size of the current structure by exactly one, thus ensuring that NEXTSUPFACT generates only minimal superfactors:

- (i) assigning labels to previously unspecified nodes;
- (ii) introducing a new neighboring node, which may either follow or precede an existing node on either tier, at both boundaries;
- (iii) add one association line, provided that both nodes are already present in the structure.

We add these superfactors to the queue in the order shown above. Consequently, Algorithm ?? will consider superfactors with new labels before superfactors with new nodes

	CV-H	CV-L	VCV-LH	CVV-LH	CVV-HL
UR	$\begin{array}{c} \text{H} \\ \\ \text{C} \quad \text{V} \end{array}$	$\begin{array}{c} \text{L} \\ \\ \text{C} \quad \text{V} \end{array}$	$\begin{array}{cc} \text{L} & \text{H} \\ & \\ \text{V} & \text{C} \quad \text{V} \end{array}$	$\begin{array}{cc} \text{L} & \text{H} \\ & \\ \text{C} & \text{V}_1 \text{V}_2 \end{array}$	$\begin{array}{cc} \text{H} & \text{L} \\ & \\ \text{V}_1 & \text{V}_2 \end{array}$
SR	–	–	–	$\begin{array}{cc} \text{L} & \text{H} \\ & \diagdown \quad \\ \text{C} & \text{V}_2 \end{array}$	$\begin{array}{cc} \text{H} & \text{L} \\ & \diagdown \quad \\ & \text{V}_2 \end{array}$

Table 3: Toy ASG

before superfactors with new association lines. Within those groups, the superfactors must also be ordered.⁵

Table 1 presents a non-exhaustive set of examples illustrating the generation of NEXTSUPFACT from an initial *deassoc* hypothesis $\alpha(x, y)$. As can be seen, each operation provides an equivalent interpretation from matrix, graph-theoretic, and logical perspectives. New quantifiers are only introduced through successor relations, corresponding to the addition of neighboring nodes in the graph structure. Within the same quantifier scope, the learner may further specialize the hypothesis through conjunction, including adding association relations (corresponding to changing matrix cells) or specifying node labels and feature values.

5. Case Study: Contour Formation

We use the contour formation to demonstrate both the logical transduction framework and BUFIA-style learning over ARs. Table 3 gives five input-output pairs, and cells marked with a dash indicate the same output as the input. The headers provide shorthand notations for each pair. The contour formation process can be observed in the CVV-LH and CVV-HL examples, where both tones become associated with the same vowel. This transformation can equivalently be represented as a change in the association matrix. Notably, tonal identities themselves are irrelevant to the deassociation and reassociation process: both CVV-LH and CVV-HL undergo contour formation once the two tones are linked to a single vowel. By contrast, an LH sequence does not necessarily surface as a contour; whether it does depends on the structural configuration of the vowels. For example, the LH sequence surfaces as a contour in CVV-LH, where the vowels are adjacent, but not in VCV-LH, where they are separated.

We will see how Algorithm ?? (LAC) finds the environment shown in Figure 6 for *deassoc*.

We call Algorithm ?? (LAC) with the data D in Table 3 letting $k = 5$. It calls Algorithm ?? (BUFIA-AC) to search for predicate *deassoc* with D, k and with $H_0 = \alpha(x, y)$.

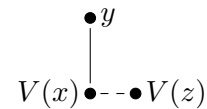


Figure 6: A distinguishing environment

5. This can be done in many ways and can impact the algorithm’s output. See [Rawski \(2021\)](#) for discussion on this point.

	(4a)	(4b)	(4c)	(4d)
New Label	$\begin{array}{c} H(y) \bullet \\ \\ x \bullet \end{array}$	$\begin{array}{c} L(y) \bullet \\ \\ x \bullet \end{array}$	$\begin{array}{c} y \bullet \\ \\ V(x) \bullet \end{array}$	$\begin{array}{c} y \bullet \\ \\ C(x) \bullet \end{array}$
New Node	$\begin{array}{c} \bullet y \\ \\ x \bullet \text{---} \bullet z \end{array}$	$\begin{array}{c} \bullet y \\ \\ z \bullet \text{---} \bullet x \end{array}$	$\begin{array}{c} z \bullet \text{---} \bullet y \\ \\ \bullet x \end{array}$	$\begin{array}{c} y \bullet \text{---} \bullet z \\ \\ x \bullet \end{array}$

 Table 4: Next Superfactors of H_0

BUFIA-AC first partitions the data into positive evidence D^+ and negative evidence D^- . In this toy example, D^+ consists of CVV-LH and CVV-HL, and D^- consists of the rest pairs marked with dashes in Table 3.

BUFIA-AC tests h_0 against both positive and negative evidence. Trivially, it is accepted by all positive examples, but, when evaluated against the negative evidence, the hypothesis is rejected: in CV-H, CV-L, and VCV-LH, the association lines in the input remain associated in the output, contrary to the prediction of h_0 . BUFIA-AC therefore concludes that the hypothesis is too general and must be made more specific.

To refine the environment, BUFIA-AC generates increasingly minimally specific structures through NEXTSUPFAC generation. Following the new label/node/association-line ordering introduced above, the learner first enques the next superfactors obtained by labeling previously unspecified nodes. These superfactors are shown in Table 4. Superfactors (4a) and (4b) label y as H or L on the tone tier, while (4c) and (4d) label x as V or C on the segment tier. Superfactors (4e) through (4h) add a single node adjacent to a current one on a tier.

When evaluating the candidate hypotheses in Table 4, BUFIA-AC determines that superfactors (4a, b, d, g) do not match all positive examples. They are thus pruned. However, (4c, e, f, h) are consistent with all positive examples, and they are thus checked against the D^- .

Each of these superfactors (4c, e, f, g) matches some negative example, hence they each fail to distinguish contexts in which the transformation applies from those in which it does not. BUFIA-AC therefore generates their next superfactors and enqueues them. BUFIA-AC then dequeues the next h , which is (4c). That this is next is a consequence of the ordering imposed by the construction of superfactors: new labels then new nodes then new associations.

Hypothesis (4c) is consistent with D^+ and so it is then checked against D^- . It is not consistent with D^- and so its superfactors are generated next. However, not all of its superfactors are enqueued. This is because the superfactors that add a label to the tonal node are pruned away by the factors in the pruned set P . The superfactors that are not pruned away are shown in Table 5, where (5a)–(5b) introduce a new node that immediately

(5a)	(5b)	(5c)	(5d)

Table 5: Next Superfactors of (4c)

precedes or follows the existing node on the segment tier, while (5c)–(5d) do so on the tone tier. These are added to the end of the queue.

The next item on the queue is (4e). It too is not consistent with D^- and its super-factors are generated and added to the queue. Note that there is some inefficiency here because the superfactor shown in Figure 7 is a next superfactor of both (4c) and (4e). Nonetheless, this procedure continues until the environment in Figure 6 is dequeued. This environment is consistent with both D^+ and D^- .

Furthermore, it is the first hypothesis in the queue to be so. To see why, consider that the environment in Figure 6 is a next superfactor of Figure 7. In other words, it is a next superfactor of a next superfactor of Hypothesis (4c). In other words, the environment in Figure 6 is generated and placed on the queue after Figure 7 is dequeued and examined for consistency.

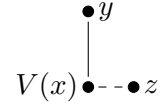


Figure 7: A superfactor generated twice

Call a next superfactor of a next superfactor a ‘grandchild.’ The ‘grandchildren’ of Hypothesis (4c) will all occur on the queue before the grandchildren of Hypotheses (4 e, f, g) by the ordering imposed. So the only way BUFIA-AC would not return the environment in Figure 6 is if a next superfactor (4 e, f, g) was consistent with both the positive and negative data. However, none of them are.

The learner therefore converges on the environment in Figure 6 as a minimal structure that triggers the transformation and is consistent with both positive and negative evidence. This formula encodes the same structural condition illustrated in Figure 5.

$$\text{Retain} = \alpha(x, y) \wedge V(x) \wedge \exists z(x \triangleleft z \wedge V(z)) \quad (4)$$

Importantly, one may observe that triggering deassociation does not require T_2 to be associated with V_2 . The learned hypothesis therefore predicts that deassociation applies whenever two vowels are adjacent, regardless of whether the following vowel is associated with a tone.

Algorithm ?? (LAC) follows a similar process to determine Add. The learner eventually converges on the following hypothesis.

$$\text{Add} = \neg\alpha(x, y) \wedge V(x) \wedge \exists z((z \triangleleft x) \wedge V(z) \wedge \alpha(z, y)) \quad (5)$$

Algorithm ?? (LAC) combines the two operations into the final transformation.

$$\begin{aligned} \phi_{\alpha(x, y)} := & \alpha(x, y) \wedge V(x) \wedge \exists z((x \triangleleft z) \wedge V(z)) \\ & \vee \neg\alpha(x, y) \wedge V(x) \wedge \exists z((z \triangleleft x) \wedge V(z) \wedge \alpha(z, y)) \end{aligned} \quad (6)$$

Lastly, the graph transduction presented in Table 2 is now complete. The licensing function ϕ_{license} filters out elements that are not associated in the surface representation. Such unlicensed elements are not realized phonetically and therefore do not appear in the output. In this way, the algorithms presented here have learned how to transform the association relation between the tonal and segmental tiers.

6. Conclusion

This work has argued that phonological processes that involve multi-tiered structures such as autosegmental representations can be uniformly captured using model theory and logical transductions. The input-output mapping can be modeled by modifying a single relation, α , through two operations of `deassoc` and `reassoc`.

We also show how BUFIA-style learning can be naturally extended to such enriched representations by traversing a partially ordered hypothesis space. A crucial component is NEXTSUPFACTOR, which generates minimally more complex structures through node labeling, the introduction of neighboring nodes, and the addition of association relations, to navigate the search space. The resulting hypothesis is a structural environment, or equivalently a logical formula, that is consistent with both the positive and negative evidence.

There are several directions for future research. One would be to characterize the class of AR transformations and prove that the above algorithms learn any transformation in this class with sufficient data. Another important direction for future work is to relax the current restriction to single deassociation and reassociation processes. Doing so will require learning more complex hypotheses involving conjunctions of structural conditions. Another limitation of the current analysis is that the transduction relies on quantification to capture across-tier dependencies. Future work can determine whether these processes can be simplified in a quantifier-free (QF) process (Strother-Garcia, 2019). This opens up more discussions about how linguistically meaningful representations can facilitate grammar inference.

References

- Steven Bird and T Mark Ellison. One-level phonology: Autosegmental representations and rules as finite automata. *Computational Linguistics*, 20(1):55–90, 1994.
- Jane Chandlee and Adam Jardine. Input and output locality and representation. *Glossa: a journal of general linguistics*, 6(1), 2021a.
- Jane Chandlee and Adam Jardine. Computational universals in linguistic theory: Using recursive programs for phonological analysis. *Language*, 93:485–519, 2021b.
- Jane Chandlee, Rémi Eyraud, Jeffrey Heinz, Adam Jardine, and Jonathan Rawski. Learning with partially ordered representations. In *Proceedings of MOL16*, pages 91–101, 2019.
- George Clements and Jay Keyser. *CV phonology: a generative theory of the syllable*. Cambridge, MA: MIT Press, 1983.
- George N. Clements and J. Goldsmith. Autosegmental studies in Bantu tone, 1984.

- John Coleman and John Local. The “no crossing constraint’ in autosegmental phonology. *Linguistics and Philosophy*, pages 295–338, 1991.
- Bruno Courcelle. Monadic second-order definable graph transductions: a survey. *Theoretical Computer Science*, 126:53–75, 04 1994.
- Bruno Courcelle and Joost Engelfriet. *Graph Structure and Monadic Second-Order Logic, a Language Theoretic Approach*. Cambridge University Press, 2012.
- Nick Danis. Logical transductions are not sufficient for notational equivalence. In *Proceedings of the 2024 Annual Meeting of Phonology*, 2025.
- Hossep Dolatian. *Computational locality of cyclic phonology in Armenian*. PhD thesis, Stony Brook University, 2020.
- Baruch Elimelech. *A Tonal Grammar of Etsako*. University of California, Los Angeles, 1976.
- Joost Engelfriet and Hendrik Jan Hoogeboom. MSO definable string transductions and two-way finite-state transducers. *ACM Transactions on Computational Logic*, 2(2):216–254, April 2001. ISSN 1529-3785.
- John Anton Goldsmith. *Autosegmental phonology*. PhD thesis, Massachusetts Institute of Technology, 1976.
- Bruce Philip Hayes. *A metrical theory of stress rules*. PhD thesis, Massachusetts Institute of Technology, 1980.
- Larry M Hyman. The representation of tone. *UC Berkeley PhonLab Annual Report*, 5(5), 2009.
- Adam Jardine. Computationally, tone is different. *Phonology*, 33(2):247–283, 2016.
- Adam Jardine. The local nature of tone-association patterns. *Phonology*, 34(2):363–384, 2017.
- Adam Jardine and Jeffrey Heinz. A concatenation operation to derive autosegmental graphs. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 2015)*, pages 139–151, 2015.
- Adam Jardine, Nick Danis, and Luca Iacoponi. A formal investigation of Q-theory in comparison to autosegmental representations. *Linguistic Inquiry*, 52(2):333–358, 2021. doi: 10.1162/ling_a_00376.
- Han Li. Learning tonotactic patterns over autosegmental representations. In *Proceedings of the Annual Meetings on Phonology*, volume 1. University of Massachusetts Amherst Libraries, 2025.
- Han Li and Jeffrey Heinz. Learning tonotactics with autosegmental representations. *Phonology*, to appear.

- Chris Oakden. Notational equivalence in tonal geometry. *Phonology*, 37(2):257–296, 2020. doi: 10.1017/S0952675720000123.
- David Odden. *Introducing Phonology*. Cambridge University Press, Cambridge, 2 edition, 2013.
- Sarah Payne. A generalized algorithm for learning positive and negative grammars with unconventional string models. In *Proceedings of the Society for Computation in Linguistics 2024*, pages 75–85, 2024.
- Jonathan Rawski. *Structure and learning in natural language*. PhD thesis, State University of New York at Stony Brook, 2021.
- Jonathan Rawski and Hossep Dolatian. Multi-input strictly local functions for tonal phonology. In *Proceedings of the Society for Computation in Linguistics 2020*, pages 208–223, 2020.
- James Rogers and Dakotah Lambert. Some classes of sets of structures definable without quantifiers. In *Proceedings of the 16th Meeting on the Mathematics of Language*, pages 63–77, Toronto, Canada, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-5706. URL <https://www.aclweb.org/anthology/W19-5706>.
- Elizabeth Caroline Sagey. *The representation of features and relations in non-linear phonology*. PhD thesis, Massachusetts Institute of Technology, 1986.
- Kristina Strother-Garcia. Imdlawn Tashlhiyt Berber syllabification is quantifier-free. In *Proceedings of the Society for Computation in Linguistics*, volume 1, 2018. Article 16.
- Kristina Strother-Garcia. *Using model theory in phonology: a novel characterization of syllable structure and syllabification*. University of Delaware, 2019.
- Kristina Strother-Garcia, Jeffrey Heinz, and Hyun Jin Hwangbo. Using model theory for grammatical inference: a case study from phonology. In Sicco Verwer, Menno van Zaanen, and Rick Smetsers, editors, *Proceedings of The 13th International Conference on Grammatical Inference*, volume 57 of *JMLR: Workshop and Conference Proceedings*, pages 66–78, October 2016.
- Logan Swanson. Learning multi tier-based strictly 2-local languages. In *Proceedings of the 18th Meeting on the Mathematics of Language*, pages 33–43, 2025.
- Logan Swanson, Jeffrey Heinz, and Jonathan Rawski. Phonotactic learning with structure, not statistics. *Linguistic Inquiry*, pages 1–24, 05 2026.
- Mai H. Vu, Ashkan Zehfroosh, Kristina Strother-Garcia, Michael Sebok, Jeffrey Heinz, and Herbert G. Tanner. Statistical relational learning with unconventional string models. *Frontiers in Robotics and AI*, Volume 5 - 2018, 2018. ISSN 2296-9144. doi: 10.3389/frobt.2018.00076.
- Bruce Wiebe. Modelling autosegmental phonology with multi-tape finite state transducers. 1992.

- Anssi Yli-Jyrä. On finite-state tonology with autosegmental representations. In *Proceedings of the 11th international conference on finite state methods and natural language processing*, pages 90–98, 2013.
- Anssi Yli-Jyrä. Three equivalent codes for autosegmental representations. In *Proceedings of the 12th International Conference on Finite-State Methods and Natural Language Processing 2015 (FSMNLP 2015 Düsseldorf)*, 2015.
- Tatevik Yolyan. A framework for learning phonological maps as logical transductions. In *Proceedings of MOL18*, pages 44–58, 2025a.
- Tatevik Arayevna Yolyan. *Phonological Expressivity and Learning via Boolean Monadic Recursive Schemes*. PhD thesis, Rutgers The State University of New Jersey, School of Graduate Studies, 2025b.